SKALA

Node 10대 · 150명이 각자 완주하는 21개 실습

쿠버네티스 심화 (집중실습)

2026.10

본 문서는 SK AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 AX



1. 환경 준비와 개인 예산
2. 실습을 시작하기 전에
3. 기초: 앱을 배포하고 밖으로 연다
4. 중급: 자원 · 배치 · 네트워크 · 볼륨
5. 심화: 상태 · 권한 · 관측 · 진단

환경 준비

150여명이 Node 10대를 함께 사용한다. 시작 전에 **두 변수**를 설정하고 1인당 예산 안에서 실습한다.

- 같은 Namespace 를 반 전체가 함께 사용한다. 내 것만 보려면 `| grep $ID` 를 덧붙인다
- 확인은 이미 떠 있는 Pod 안에서 `kubectl exec` 로 한다. 임시 Pod 는 13번에서만 만든다

```
# ① 터미널을 새로 열 때마다
export NS=<반 Namespace>              # class-1 ~ class-10
export ID=<본인 식별자>               # 소문자와 숫자만 (예: sk001)
kubectl config set-context --current --namespace=$NS
# ② 실습 파일을 만든다 (handson-lab-2610 폴더에서 한 번만)
./render.sh                           # front.conf · ing-$ID.yaml 등 7개
# ③ 내 Pod 수를 센다. 3을 넘으면 정리한 뒤 다음 실습으로 간다
kubectl get pod -n $NS --no-headers | grep -c "^$ID-"
kubectl get pvc -n $NS | grep $ID
```

구분	범위
개인 별 범위	**Pod 3개 · PVC 1개. 모든 실습이 이 안에서 끝난다**
만들 수 있는 것	워크로드 · Service · Ingress · PVC · ConfigMap · Secret · HPA · Role · NetworkPolicy
조회 · 권한 밖	조회는 클러스터 전체, 수정은 자기 반 Namespace 까지. Node 조작 · CRD · ClusterRole 은 권한 X

[주의] 모든 리소스 이름과 Ingress 경로 앞에 `$ID` 를 붙인다. 어기면 같은 반 동료의 것을 덮어쓴다.

1. 기초: 앱을 배포하고 밖으로 연다

- **backend** 와 **frontend** 를 만들고 Service 로 서로 **연결한다**
- **Ingress** 로 **외부에서의 접속을 구성한다.**
- **여기서 만든 것을 중급 · 심화에서 계속 다시 수정한다.**

본 문서는 SK AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 AX

[기초 1] backend 를 배포한다

앱의 뒷단부터 만들면서 **Deployment** 를 거치는 이유를 직접 확인한다.

- Deployment → ReplicaSet → Pod
- `nginx-hello` 는 요청을 받은 Pod 이름을 `Server name` 으로 돌려준다. 뒤에서 계속 사용한다

```
# ① backend 를 배포한다. 이 이미지는 8080 포트에서 응답한다
kubectl create deploy $ID-back --image=nginxdemos/nginx-hello:plain-text -n $NS
kubectl get deploy,rs,pod -n $NS | grep $ID-back

# ② 이름이 세 층으로 이어진다
#   deploy/$ID-back  →  rs/$ID-back-6d4f  →  pod/$ID-back-6d4f-aa1

# ③ 개수를 2로 바꾼다. ReplicaSet 은 몇 개가 되는가?
kubectl scale deploy/$ID-back --replicas=2 -n $NS
kubectl get rs -n $NS | grep $ID-back

# ④ Pod 를 하나 지운다. 다시 2개가 되는가?
kubectl get pod -n $NS -l app=$ID-back
kubectl delete pod <Pod 이름> -n $NS
kubectl get pod -n $NS -l app=$ID-back
```

[기초 2] Pod 안을 들여다본다

장애의 첫 10분은 대부분 **로그 · 셸 · 포트** 세 가지로 끝난다. 그 세 경로를 손에 익힌다.

- Service 도 Ingress 도 없이 `port-forward` 로 내 PC 에서 호출할 수 있다

```
# ① 로그를 실시간으로 본다
kubectl logs -f deploy/$ID-back -n $NS          # Ctrl+C 로 빠져나온다

# ② 컨테이너 안에 들어간다
kubectl exec -it deploy/$ID-back -n $NS -- sh
#   안에서:  id  ·  env | sort | head  ·  exit

# ③ 내 PC 로 포트를 당겨 직접 호출한다
kubectl port-forward -n $NS deploy/$ID-back 8080:8080
#   다른 터미널에서:  curl localhost:8080

# ④ 응답의 Server name 이 Pod 이름이다
#   Server name: $ID-back-6d4f-aa1

# ⑤ 종료된 컨테이너의 로그는 --previous 로만 볼 수 있다
kubectl logs deploy/$ID-back -n $NS --previous   # 재시작 이력이 없으면 에러가 정상
```

[기초 3] Service 로 이름을 준다

Pod IP 는 언제든 바뀐다. **이름**을 붙이면 호출하는 쪽이 바뀌지 않는다.

- 대상을 고르는 것은 이름이 아니라 **라벨**이다
- EndpointSlice 가 비어 있으면 트래픽이 갈 곳이 없다. 오류는 발생하지 않는다

```
# ① Service 를 만든다. 80 으로 받아 컨테이너의 8080 으로 넘긴다
kubectl expose deploy/$ID-back --name=$ID-back --port=80 --target-port=8080 -n $NS
kubectl get svc $ID-back -n $NS

# ② 뒤에 연결된 Pod 목록을 본다. 여기가 비면 안 된다
kubectl get endpointslice -n $NS -l kubernetes.io/service-name=$ID-back

# ③ backend 안에서 이름으로 여섯 번 호출한다. Server name 이 바뀌는가?
kubectl exec deploy/$ID-back -n $NS -- sh -c \
  "for i in 1 2 3 4 5 6; do curl -s http://$ID-back/ | grep 'Server name'; done"

# ④ Selector 를 일부러 틀린다. 오류가 발생하는가?
kubectl set selector svc/$ID-back app=$ID-none -n $NS
kubectl get endpointslice -n $NS -l kubernetes.io/service-name=$ID-back
kubectl set selector svc/$ID-back app=$ID-back -n $NS     # 되돌린다
```

[성공] ③에서 `Server name` 이 둘로 갈리면 성공이다. ④처럼 ENDPOINTS 가 비면 어디부터 보겠는가?

[기초 4] frontend 를 추가한다

앞단을 세워 backend 를 **이름으로** 호출하게 하고, 설정은 ConfigMap 으로 넣는다.

- frontend 는 **Service 이름만** 안다
- 설정을 밖으로 분리하면 같은 이미지로 모든 환경을 실행한다

```
# ① frontend 설정. render.sh 가 front.conf 를 만들어 두었다
#   location /api/ { proxy_pass http://$ID-back/; }    ← 이름으로 호출한다
kubectl create cm $ID-front-conf -n $NS --from-file=default.conf=front.conf

# ② frontend 를 배포하고 설정을 마운트한다
kubectl create deploy $ID-front --image=nginx:1.27 -n $NS
kubectl patch deploy $ID-front -n $NS -p '{"spec":{"template":{"spec":{"volumes":
  [{"name":"conf","configMap":{"name":"'$ID'-front-conf"}}],
  "containers":[{"name":"nginx",
   "volumeMounts":[{"name":"conf","mountPath":"/etc/nginx/conf.d"}]}]}}}}'
kubectl expose deploy/$ID-front --name=$ID-front --port=80 -n $NS
kubectl rollout status deploy/$ID-front -n $NS

# ③ frontend 안에서 호출한다. 두 응답이 어떻게 다른가?
kubectl exec deploy/$ID-front -n $NS -- curl -s localhost/
kubectl exec deploy/$ID-front -n $NS -- curl -s localhost/api/
```

[성공] ③에서 `front OK` 와 `Server name` 이 각각 나오면 성공이다. frontend 는 backend 의 무엇을 아는가?

[기초 5] 바뀐 설정을 반영한다

ConfigMap 을 수정해도 Pod 가 **아무 일도 하지 않는** 이유와 반영 방법을 확인한다.

- ConfigMap 을 수정해도 Pod 템플릿은 그대로이다

```
# ① 응답 문구를 바꾼다
sed -i.bak "s/front OK/front v2/" front.conf
kubectl create cm $ID-front-conf -n $NS --from-file=default.conf=front.conf \
  --dry-run=client -o yaml | kubectl apply -f -

# ② 1~2분 뒤, Pod 안의 파일은 바뀌었는가? 응답은?
kubectl exec deploy/$ID-front -n $NS -- grep front /etc/nginx/conf.d/default.conf
kubectl exec deploy/$ID-front -n $NS -- curl -s localhost/

# ③ 강제로 다시 기동한다
kubectl rollout restart deploy/$ID-front -n $NS
kubectl rollout status deploy/$ID-front -n $NS
kubectl exec deploy/$ID-front -n $NS -- curl -s localhost/

# ④ 이력을 보고 되돌려 본다
kubectl get rs -n $NS -l app=$ID-front
kubectl rollout undo deploy/$ID-front -n $NS
```

[성공] ③ 뒤에 `front v2` 가 나오면 여기까지 온 것이다. ②에서 파일은 바뀌었는데 응답은 왜 그대로였는가?

[기초 6] Ingress 로 밖에 연다

클러스터 안에서만 닿던 앱에 **Ingress** 를 추가해 밖에서 접속되는 데까지 간다.

- 경로는 `/$ID` 로 나눈다. Ingress Controller 를 150명이 **함께 사용하기** 때문이다
- 경로가 긴 규칙이 먼저 적용된다. `/$ID/api` 가 `/$ID` 보다 우선한다
- `rewrite-target` 이 경로를 `/` 로 바꿔 넘긴다. backend 응답의 `URI` 로 확인한다

```
# ① Ingress 를 만든다 (render.sh 가 만든 ing-$ID.yaml)
#   kind: Ingress · name: $ID-app · ingressClassName: public-nginx
#   annotations: nginx.ingress.kubernetes.io/rewrite-target: /
#   paths(Prefix):  /$ID/api → $ID-back:80   ·   /$ID → $ID-front:80
kubectl apply -f ing-$ID.yaml -n $NS

# ② ADDRESS 가 표시되면 변수에 저장한다. 비어 있으면 잠시 기다린다
kubectl get ingress $ID-app -n $NS
ING=$(kubectl get ingress $ID-app -n $NS -o jsonpath='{..hostname}')

# ③ 밖에서 호출한다
curl -s http://$ING/$ID/
curl -s http://$ING/$ID/api/ | grep -E 'Server name|URI'
```

[성공] ③의 두 경로가 각각 front 와 back 으로 가면 성공이다. back 이 받은 `URI` 는 왜 `/` 인가?

[기초 7] 선언형으로 관리한다

명령으로 만든 것은 기록이 남지 않지만, **파일로 옮기면** 리뷰도 되돌리기도 된다.

- `apply` 는 '이 상태여야 한다' 를 보낸다. 적용 전에 `diff` 로 먼저 본다

```
# ① 지금 상태를 파일로 추출한다
kubectl get deploy $ID-front -n $NS -o yaml > front.yaml
#   지울 것: status · metadata.uid · resourceVersion · creationTimestamp

# ② 파일을 수정한다. metadata.labels 아래에 한 줄을 추가한다  ·  vi front.yaml
#     tier: front

# ③ 적용 전에 차이를 본다. 이 습관이 사고를 막는다
kubectl diff -f front.yaml -n $NS

# ④ 적용한다
kubectl apply -f front.yaml -n $NS
kubectl get deploy $ID-front -n $NS --show-labels

# ⑤ 같은 파일을 한 번 더 적용한다. 무슨 일이 일어나는가?
kubectl apply -f front.yaml -n $NS
```

[성공] ④가 configured, ⑤가 unchanged 면 된 것이다. 몇 번을 적용해도 결과가 같은 이유를 설명해 본다.

2. 중급: 자원 · 배치 · 네트워크 · 볼륨

- 기초에서 만든 앱에 숫자와 제약을 더한다
- 요청이 **Ingress 에서 Pod 까지** 어떻게 가는지 끝까지 추적한다
- backend 로 들어오는 통신을 제한하고 볼륨을 연결한다

본 문서는 SK AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 AX

[중급 8] 숫자를 측정으로 정한다

지금 앱에는 `requests` 가 없다. 적지 않으면 스케줄러가 0으로 보고 Node 가 가득 차도 계속 배치한다.

- `requests` 는 자리 예약, `limits` 는 상한. Node 대수를 정하는 것은 **`requests`** 이다
- `kubectl top` 은 그 순간 값이다. 한 번으로 정하지 않는다

```
# ① 지금은 비어 있다. 감으로 한 번 적는다
kubectl set resources deploy/$ID-back -n $NS \
  --requests=cpu=50m,memory=32Mi --limits=cpu=100m,memory=64Mi

# ② QoS 등급이 무엇으로 정해졌는지 본다
kubectl get pod -n $NS -l app=$ID-back \
  -o custom-columns=NAME:.metadata.name,QOS:.status.qosClass

# ③ 다른 터미널에서 frontend 를 통해 2분간 요청을 보낸다 (임시 Pod 없이)
kubectl exec deploy/$ID-front -n $NS -- sh -c \
  'for i in $(seq 600); do curl -s -o /dev/null localhost/api/; sleep 0.2; done'

# ④ 10초 간격으로 12번 샘플링하고, 최댓값을 ①의 감과 비교한다
for i in $(seq 12); do \
  kubectl top pod -n $NS --no-headers | grep $ID-back; sleep 10; done | tee top.log
sort -k2 -n top.log | tail -1 ; sort -k3 -n top.log | tail -1
```

[성공] `top.log` 에 24줄(Pod 2개 × 12번)이 쌓여야 한다. 최댓값과 ①에서 적은 값이 몇 배 차이인지 적어 둔다.

[중급 9] 한 Node 에 몰리지 않게 한다

backend Pod 2개를 **서로 다른 Node** 에 배치해, Node 하나가 멈춰도 서비스가 남는 구성을 만든다.

- 제약을 지킬 수 없으면 배치하지 않는다. `Pending` 은 정의대로 동작한 것이다

```
# ① 지금 어느 Node 에 있는가
kubectl get pod -n $NS -l app=$ID-back -o wide

# ② 고르게 배치하는 제약을 적용한다. 다시 ①을 실행하면 Node 가 갈리는가?
kubectl patch deploy $ID-back -n $NS --type=merge -p '{"spec":{"template":{"spec":
  {"topologySpreadConstraints":[{"maxSkew":1,
   "topologyKey":"kubernetes.io/hostname","whenUnsatisfiable":"DoNotSchedule",
   "labelSelector":{"matchLabels":{"app":"'$ID'-back"}}}]}}}}'

# ③ 지킬 수 없는 제약을 더한다. 존재하지 않는 zone 을 요구한다
kubectl patch deploy $ID-back -n $NS --type=merge -p '{"spec":{"template":{"spec":
  {"nodeSelector":{"topology.kubernetes.io/zone":"ap-northeast-2z"}}}}}'
kubectl get pod -n $NS -l app=$ID-back          # 새 Pod 는 Pending, 기존 2개는 그대로
kubectl describe pod -n $NS -l app=$ID-back | grep -A2 FailedScheduling

# ④ 되돌린다. ②의 제약은 남는다
kubectl rollout undo deploy/$ID-back -n $NS
```

[성공] ③에서 새 Pod 만 `Pending` 이고 기존 2개가 `Running` 이면 성공이다. 서비스가 끊기지 않은 이유는?

[중급 10] 부하에 따라 확장한다

사람이 밤에 깨어 Pod 를 늘릴 수는 없다. HPA 는 `requests` 를 기준으로 계산하므로 **8번의 숫자**가 맞아야 한다.

- 목표 사용률은 `limits` 가 아니라 **`requests` 대비 비율**이다. 여기서 자주 틀린다
- 개인 예산에 맞춰 `--max=2` 로 제한한다. 운영에서는 늘어난 Pod 가 배치될 Node 여유까지 계산한다

```
# ① backend 를 1개로 줄이고 HPA 를 만든다 (requests 가 있어야 동작한다)
kubectl scale deploy/$ID-back --replicas=1 -n $NS
kubectl autoscale deploy/$ID-back -n $NS --cpu-percent=50 --min=1 --max=2

# ② 지표를 읽는다. TARGETS 가 <unknown> 이면 requests 가 없는 것이다
kubectl get hpa $ID-back -n $NS -w              # TARGETS 열을 본다, Ctrl+C

# ③ 다른 터미널에서 backend 안의 CPU 를 150초간 사용한다
kubectl exec deploy/$ID-back -n $NS -- timeout 150 sh -c 'while :; do :; done'

# ④ 늘어나는 과정을 본다. 계산식은 describe 에 그대로 나온다
kubectl get hpa $ID-back -n $NS ; kubectl get pod -n $NS -l app=$ID-back
kubectl describe hpa $ID-back -n $NS | tail -8

# ⑤ 정리한다. 뒤 실습은 backend 1개로 진행한다
kubectl delete hpa $ID-back -n $NS ; kubectl scale deploy/$ID-back --replicas=1 -n $NS
```

[성공] `TARGETS` 가 200% 근처로 오르고 Pod 가 2개가 되면 성공이다. ③의 exit code 143 은 정상이다.

[중급 11] 메모리를 초과하면 종료된다

메모리를 계속 차지하는 버전을 배포한 상황을 만든다. **종료 코드와 이벤트만으로** 원인을 찾는다.

- 종료 코드 137 = SIGKILL. 커널이 종료시켰으므로 앱 로그에는 흔적이 없다
- readiness Probe 가 없으면 기동 직후 Ready 로 처리된다. 기존 Pod 가 내려가 **서비스가 끊긴다**

```
# ① 메모리를 끝없이 차지하는 명령으로 바꿔 배포한다 (limits 는 64Mi)
kubectl patch deploy $ID-back -n $NS --type=json -p '[{"op":"add",
  "path":"/spec/template/spec/containers/0/command",
  "value":["sh","-c","tail /dev/zero"]}]'

# ② 새 Pod 의 재시작이 쌓인다. 기존 Pod 는 남아 있는가? 서비스는 응답하는가?
kubectl get pod -n $NS -l app=$ID-back -w      # RESTARTS 증가, Ctrl+C
kubectl exec deploy/$ID-front -n $NS -- curl -sI localhost/api/ | head -1

# ③ 종료 사유를 읽는다. 여기에 답이 있다
kubectl describe pod -n $NS -l app=$ID-back | grep -A3 'Last State'

# ④ 종료 직전의 로그. 앱이 남긴 오류가 있는가?
kubectl logs -n $NS -l app=$ID-back --previous --tail=20
# ⑤ 이전 버전으로 되돌린다. 빠뜨리면 다음 실습이 전부 막힌다
kubectl rollout undo deploy/$ID-back -n $NS
```

[성공] ③에 `OOMKilled` 와 137, ②에 502 가 보이면 여기까지 온 것이다. ⑤ 복구를 잊지 않는다.

다섯 개 홉을 그려 본다

연결되지 않는다는 문의는 **어느 홉에서 끊겼는지**만 알면 절반이 풀린다.



끊긴 지점 앞뒤로 나누면 볼 곳이 절반으로 준다

[중급 12] 요청이 닿는 전 구간

다섯 홉을 하나씩 건너뛰며 호출해, 어느 구간까지 응답하는지로 끊긴 지점을 찾는다.

- Ingress → front Service → front Pod → back Service → back Pod
- ①이 막히고 ③이 되면 Ingress, ③이 막히고 ④가 되면 frontend 설정이다

```
# ① 밖에서 (Ingress 를 통과하는가)
ING=$(kubectl get ingress $ID-app -n $NS -o jsonpath='{..hostname}')
curl -s -o /dev/null -w "%{http_code}\n" http://$ING/$ID/api/

# ② Ingress 규칙이 어느 Service 를 가리키는가
kubectl describe ingress $ID-app -n $NS | grep -A5 Rules

# ③ 클러스터 안에서 front Service 로 (Ingress 를 건너뛴다)
kubectl exec deploy/$ID-back -n $NS -- \
  curl -s -o /dev/null -w "%{http_code}\n" http://$ID-front/api/

# ④ back Service 로 직접 (frontend 를 건너뛴다)
kubectl exec deploy/$ID-front -n $NS -- curl -s http://$ID-back/ | grep 'Server name'

# ⑤ 각 Service 뒤에 Pod 가 연결되어 있는가
kubectl get endpointslice -n $NS | grep $ID
```

[성공] ①③④가 모두 응답하면 성공이다. ①만 안 되고 ③은 되면 어디를 보겠는가?

[중급 13] 통신을 제한한다

지금은 아무 Pod 나 backend 에 직접 닿는다. 내 backend 에 **frontend 만** 들어오도록 제한한다.

- 정책이 하나라도 적용되면 그 Pod 는 **기본 거부**로 바뀐다
- `podSelector` 는 `app: $ID-back` 으로 좁힌다. 비워 두면 반 전체 Pod 가 막힌다

```
# ① 지금은 아무 Pod 나 backend 에 닿는다. 임시 Pod 하나로 확인한다 (200)
kubectl run $ID-net --rm -i --restart=Never --image=curlimages/curl -n $NS -- \
  curl -s -m 3 -o /dev/null -w "%{http_code}\n" http://$ID-back/

# ② frontend 에서 오는 것만 허용한다 (render.sh 가 만든 netpol-$ID.yaml)
#   podSelector.matchLabels: {app: $ID-back}
#   ingress[0].from[0].podSelector.matchLabels: {app: $ID-front}
kubectl apply -f netpol-$ID.yaml -n $NS

# ③ ①을 다시 실행한다. 3초 뒤 000 과 함께 실패하면 막힌 것이다

# ④ frontend 를 거치면 여전히 응답한다
kubectl exec deploy/$ID-front -n $NS -- curl -s -m 3 localhost/api/ | grep 'Server name'

# ⑤ 정책을 지운다. 남겨 두면 Ingress 의 /$ID/api 경로도 막힌다
kubectl delete -f netpol-$ID.yaml -n $NS
```

[성공] ③은 막히고 ④는 되면 성공이다. 막는 것이 목적이었으니 끝나면 ⑤로 정책을 지운다.

[중급 14] 볼륨을 연결한다

컨테이너는 사라진다. **남겨야 할 것**이 생기면 볼륨을 연결한다.

- PVC 가 `Pending` 인 것이 정상일 때가 있다. Pod 가 연결되어야 볼륨이 만들어지는 방식이다
- PVC 예산은 1인당 1개이다. 여기서 지우지 않으면 다음 DB 실습에서 볼륨을 받지 못한다

```
# ① 볼륨을 신청한다 (pvc-$ID.yaml · 1Gi · ReadWriteOnce)
kubectl apply -f pvc-$ID.yaml -n $NS
kubectl describe pvc $ID-data -n $NS | tail -3       # Pending. 이유를 읽는다
# ② Pod 를 연결한다. 그제서야 볼륨이 만들어진다
kubectl apply -f pod-$ID.yaml -n $NS
kubectl get pvc $ID-data -n $NS -w                   # Bound 가 되면 Ctrl+C

# ③ 파일을 남기고 Pod 를 지운다
kubectl exec $ID-vol -n $NS -- sh -c 'date > /data/memo; cat /data/memo'
kubectl delete pod $ID-vol -n $NS

# ④ Pod 를 다시 만들어 같은 파일을 읽는다
kubectl apply -f pod-$ID.yaml -n $NS                 # Running 이 된 뒤 다음 줄
kubectl exec $ID-vol -n $NS -- cat /data/memo
# ⑤ 정리한다. Pod 와 PVC 를 모두 지운다
kubectl delete -f pod-$ID.yaml -f pvc-$ID.yaml -n $NS
```

[성공] ④에서 ③과 같은 시각이 나오면 성공이다. ①의 `Pending` 은 고장이 아니다.

3. 심화: 상태 · 권한 · 관측 · 진단

- DB 를 추가해 **상태 있는 구성**으로 바꾸고 백업 · 복원까지 한다
- 권한과 컨테이너 보안을 직접 제한한다
- 증상에서 원인까지 좁혀 가는 순서를 몸에 익힌다

본 문서는 SK AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 AX

[심화 15] DB 를 추가한다

앱에 상태를 더한다. **DB 처럼 개체를 구분**해야 하면 Deployment 로는 부족하고, 이름과 볼륨이 고정돼야 한다.

- StatefulSet 의 Pod 는 `-0` 처럼 안정된 이름을 갖고, 재시작해도 **자기 PVC 를 다시 찾는다**
- 복제본은 1개이다. backend 1 · frontend 1 · DB 1 로 Pod 3개가 채워진다

```
# ① DB 를 만든다 (sts-$ID.yaml)
#   ConfigMap(초기 SQL 100행) · headless Service · StatefulSet $ID-db 복제 1
kubectl apply -f sts-$ID.yaml -n $NS
kubectl rollout status sts/$ID-db -n $NS --timeout=180s

# ② 이름과 PVC 를 본다. 규칙이 보이는가?
kubectl get pod,pvc -n $NS | grep $ID-db

# ③ 한 행을 추가하고 Pod 를 지운다. 이름 · 볼륨 · 데이터가 어떻게 되는가?
kubectl exec $ID-db-0 -n $NS -- psql -U app -c "insert into t(note) values('mine');"
kubectl delete pod $ID-db-0 -n $NS
kubectl get pod,pvc -n $NS | grep $ID-db        # Running 이 된 뒤 다음 줄
kubectl exec $ID-db-0 -n $NS -- psql -U app -tAc 'select count(*) from t;'     # 101
# ④ Pod 별 주소로 호출할 수 있다 (headless Service)
kubectl exec deploy/$ID-front -n $NS -- getent hosts $ID-db-0.$ID-db
```

[성공] ③에서 **같은 이름 · 같은 PVC** 로 기동하고 101 이 나오면 된 것이다. Deployment 였다면?

[심화 16] 지우고 되살린다

백업은 **복원을 해 본 적이 있을 때만** 백업이다. 지워 보고 되살리는 것까지가 한 벌이다.

- 원본은 덮어쓰지 않는다. **새 DB 에 복원**해 검증한 뒤 교체한다
- 논리 백업은 DB 가 살아 있어야 받을 수 있다. 볼륨이 깨진 상황은 복구하지 못한다

```
# ① 지금 데이터를 확인한다 (101)
kubectl exec $ID-db-0 -n $NS -- psql -U app -tAc 'select count(*) from t;'

# ② 백업을 받는다. 크기가 0이면 실패한 것이다
kubectl exec $ID-db-0 -n $NS -- pg_dump -U app -Fc app > $ID.dump
ls -lh $ID.dump

# ③ 사고를 낸다. 되돌릴 수 없다
kubectl exec $ID-db-0 -n $NS -- psql -U app -c 'truncate t;'
kubectl exec $ID-db-0 -n $NS -- psql -U app -tAc 'select count(*) from t;'

# ④ 원본 app 은 그대로 두고, 새 DB restored 에 복원한다
kubectl exec $ID-db-0 -n $NS -- createdb -U app restored
kubectl exec -i $ID-db-0 -n $NS -- pg_restore -U app -d restored < $ID.dump
kubectl exec $ID-db-0 -n $NS -- psql -U app -d restored -tAc 'select count(*) from t;'
```

[성공] ④의 건수가 ①과 같으면 성공이다. 원본을 그대로 둔 채 복원한 까닭을 적어 본다.

[심화 17] 준비되기 전에 보내지 않는다

liveness 에 포트를 잘못 적은 상황을 만든다. Probe 는 **잘 사용하면 무중단을, 잘못 사용하면 장애를** 만든다.

- readiness 는 트래픽만 끊고, liveness 는 **컨테이너를 종료**한다
- 기동이 느린 앱에는 `startupProbe`

```
# ① liveness 에 Service 포트(80)를 적는다. 컨테이너는 8080 에서 듣는다
kubectl patch deploy $ID-back -n $NS -p '{"spec":{"template":{"spec":
  {"containers":[{"name":"nginx-hello","livenessProbe":
   {"httpGet":{"path":"/","port":80},"periodSeconds":3,"failureThreshold":2}}]}}}}'

# ② 재시작이 쌓이는지 본다
kubectl get pod -n $NS -l app=$ID-back -w        # Ctrl+C

# ③ 종료 원인을 이벤트로 읽는다
kubectl describe pod -n $NS -l app=$ID-back | grep -i -A3 'liveness\|Last State'

# ④ 포트를 8080 으로 수정하고 간격을 현실적으로 정한다
kubectl patch deploy $ID-back -n $NS -p '{"spec":{"template":{"spec":
  {"containers":[{"name":"nginx-hello","livenessProbe":
   {"httpGet":{"path":"/","port":8080},"periodSeconds":10,"failureThreshold":3}}]}}}}'
```

[성공] ③에서 liveness 실패를 확인했으면 끝이고, 일부러 낸 고장이니 ④로 복구한다.

[심화 18] 권한을 제한한다

기본 ServiceAccount 는 생각보다 넓고, 의도한 것이 실제로 막히는지는 **명령으로만** 확인할 수 있다.

- 매니페스트를 눈으로 보고 판단하지 않는다. `auth can-i` 로 검증한다
- **no 가 나오는 것**이 이 실습의 산출물이다

```
# ① 지금 내 권한이 어디까지인가
kubectl auth can-i --list -n $NS | head -20

# ② SA · Role · RoleBinding 을 만든다 (rbac-$ID.yaml)
#   SA: $ID-dev · Role: pods/services 에 get,list,watch 만 · RoleBinding 으로 연결
kubectl apply -f rbac-$ID.yaml -n $NS

# ③ 그 SA 의 토큰으로 검증한다
TOKEN=$(kubectl create token $ID-dev -n $NS)
kubectl auth can-i get pods         -n $NS --token=$TOKEN   # yes
kubectl auth can-i get secrets      -n $NS --token=$TOKEN   # no
kubectl auth can-i create pods/exec -n $NS --token=$TOKEN   # no

# ④ Pod 가 SA 로 API 를 호출할 때 무엇이 주입되는가
kubectl exec deploy/$ID-back -n $NS -- ls /var/run/secrets/kubernetes.io/serviceaccount
```

[심화 19] 컨테이너를 가둔다

컨테이너는 **기본값이 느슨하다**. 네 가지만 채워도 크게 좁혀진다.

- `runAsNonRoot` · `allowPrivilegeEscalation: false` · `drop [ALL]` · `seccompProfile`
- 이미지가 root 가 아닌 계정으로 실행되도록 만들어져 있어야 한다. 설정만으로는 바뀌지 않는다

```
# ① 실행 계정을 비교한다. frontend 는 root 이다. backend 는?
kubectl exec deploy/$ID-front -n $NS -- id
kubectl exec deploy/$ID-back  -n $NS -- id

# ② backend 에 네 가지를 채운다
kubectl patch deploy $ID-back -n $NS -p '{"spec":{"template":{"spec":
  {"securityContext":{"runAsNonRoot":true,"runAsUser":101,
   "seccompProfile":{"type":"RuntimeDefault"}},
   "containers":[{"name":"nginx-hello","securityContext":
   {"allowPrivilegeEscalation":false,"capabilities":{"drop":["ALL"]}}}]}}}}'
kubectl rollout status deploy/$ID-back -n $NS

# ③ 실제로 적용됐는지 프로세스 상태로 확인한다
kubectl exec deploy/$ID-back -n $NS -- grep -E 'CapEff|NoNewPrivs' /proc/1/status

# ④ 같은 설정을 frontend 에 적용하면 기동하는가? 적용하지 말고 이유를 적는다
```

[성공] ③에서 `CapEff` 가 0, `NoNewPrivs` 가 1 이면 된 것이다. ④의 답은 ①의 `id` 결과에 있다.

[심화 20] 지표에서 로그로 좁힌다

전체 → 내 Pod → 로그. 순서가 곧 진단의 과정이다.

- 전체 평균은 문제를 감춘다. Pod 별로 나누면 하나만 높은 것이 보인다

```
# ① 반 전체를 먼저 본다. 누가 많이 사용하는가
kubectl top pod -n $NS --sort-by=cpu | head -6

# ② 내 것만 추린다. 지금은 셋 다 조용하다
kubectl top pod -n $NS --containers | grep $ID

# ③ 다른 터미널에서 backend 의 CPU 를 90초간 사용하고, 1분 뒤 ②를 다시 본다
kubectl exec deploy/$ID-back -n $NS -- timeout 90 sh -c 'while :; do :; done'

# ④ 값이 높은 Pod 의 로그만 시간대로 좁힌다
kubectl logs -n $NS deploy/$ID-back --since=10m --tail=20

# ⑤ 라벨로 묶어 여러 Pod 의 로그를 한 번에 볼 수도 있다
kubectl logs -n $NS -l app=$ID-back --since=10m --prefix | tail -30
```

[성공] ②에서 backend 만 100m 근처로 올라가면 된 것이다. 응답이 느리다는 문의에 어느 순서로 보겠는가?

[심화 21] 단서가 없을 때

여기까지 만든 앱을 한 순서로 점검하면, 단서가 없어도 **볼 곳이 다섯 군데**로 좁혀진다.

- 수정하기 전에 **끊긴 지점부터** 특정한다. 수정해서 나아지면 원인을 모른 채 넘어간다
- 산출물은 **증상 · 원인 · 근거** 세 줄이다. 다음에 재현할 수 있어야 진단이다
- ②는 12번에서 익힌 순서 그대로이다

```
# ① 상태부터. STATUS 와 READY 가 갈림길이다
kubectl get pod -n $NS | grep $ID
#   Pending → 배치 실패   CrashLoop → 기동 실패   Running 0/1 → readiness

# ② 밖에서 안으로 한 홉씩 (12번의 순서 그대로)
ING=$(kubectl get ingress $ID-app -n $NS -o jsonpath='{..hostname}')
curl -s -o /dev/null -w "%{http_code}\n" http://$ING/$ID/api/
kubectl get ingress,svc,endpointslice -n $NS | grep $ID

# ③ 최근에 무슨 일이 있었는지
kubectl get events -n $NS --sort-by=.lastTimestamp | grep $ID | tail -12

# ④ 배포가 진행 중인지 멈췄는지
kubectl rollout status deploy/$ID-back -n $NS --timeout=10s

# ⑤ 그래도 모르면 로그로
kubectl logs -n $NS -l app=$ID-back --since=15m --prefix | tail
```